Stack3 Security — Project Blueprint

29 October 2026 · Hacker House Goa 2026 · Imran, Yash, Ivo

Overview

Stack3 Security protects Web3 developers before they run code and users before they sign.

Two separate products. The developer side and the consumer side are different products for different people, built and installed separately. They share only the backend API and the threat graph.

	Developer product	Consumer product
Who uses it	Developers cloning, reviewing or building repos	Anyone using dApps and wallets in Chrome
Where it runs	VS Code extension + `npx stack3` CLI	Chrome extension
What it protects	The developer's machine, keys and project	The user's wallet and seed phrase
Main checks	Malicious repo files, malicious packages, leaked secrets	Seed-phrase phishing pages, risky signing requests, phishing domains, bad page scripts
Owner (logic)	Imran	Yash
Screens	Ivo	Ivo

Pitch line: "The website was real. Its code wasn't." In December 2023 a hijacked npm package (Ledger Connect Kit) turned real dApp websites into wallet drainers, and domain blocklists missed it because the domains were genuine.

The link between the two products: when the developer side finds an attacker wallet or domain inside a malicious repo or package, it is written to the graph. When a user is about to send funds or grant permissions to that same wallet, the Chrome extension warns them.

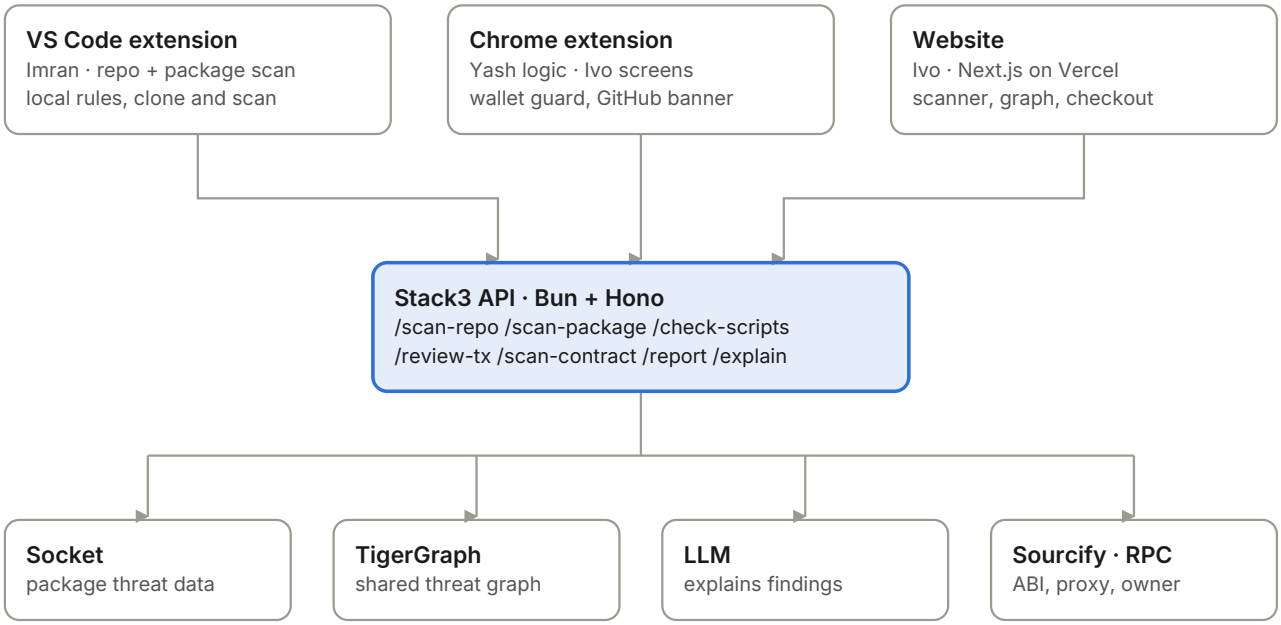
Changes from the first plan (Stack3 PDF)

- The developer side now checks **malicious repo files** (`.vscode/tasks.json`, `.vscode/settings.json`, `package.json` install scripts), not only packages. This is how fake-interview attacks on Web3 developers work.
- The VS Code extension **scans before a folder is opened** ("Clone and scan safely"), not after a Web3 workspace is already open.
- An **AI explanation layer** is added. Rules decide the verdict; AI only explains it. The event theme is AI × Crypto.
- Added: **page-script check** on dApps, **GitHub repo banner** in Chrome, **CLI** for terminal, Cursor and private repos.
- Added: **page scan** in the Chrome extension. It reads the whole code of each page (HTML, forms, loaded scripts) and warns as soon as a site asks for a seed phrase or private key, before the user types anything.
- **Wallet interception moves from the frontend owner to Yash**, who owns the consumer logic.
- **Cut:** server-side Slither worker, GoPlus, Honeypot.is, Seaport decoding, deployer-funding indexing. Transaction simulation becomes a stretch goal.
- The numeric risk score is replaced by **verdict levels**.
- The demo flags the fake package **when it is added, before install**, and the fake package stays **local, never published to npm**.

Architecture

All three clients call one API. The detection rules live in a shared TypeScript `engine` package, so the VS Code extension and the CLI can run repo-file rules locally, without sending code to the server.

Three clients share one API and one threat graph



The API is the only thing that talks to Socket, TigerGraph, the LLM and the chain. Dodo Payments checkout runs from the website through a Next.js route, outside this API.

Monorepo layout

```
stack3/
├── packages/
│   ├── engine/   ← rules, verdict format, parsers (Imran core, Yash consumer rules)
│   ├── ui/       ← shared React components (Ivo)
│   └── cli/      ← npx stack3 scan / check (Imran)
├── apps/
│   ├── api/      ← Bun + Hono, one route file per owner
│   ├── vscode/   ← VS Code extension (Imran logic, Ivo panels)
│   ├── chrome/   ← Chrome extension (Yash logic, Ivo screens)
│   ├── web/      ← Next.js website (Ivo)
│   └── demo/     ← demo sites and fake package, local only (Ivo + Imran)
└── graph/        ← TigerGraph schema and queries (Yash)
```

Team and ownership

Each person owns their own logic and their own API route files. Nobody edits another person's files without asking.

Member	Owns	Main deliverables	API routes
Imran	Developer side + shared engine	VS Code extension logic, repo-file rules, Socket package scan, secret scan, CLI, AI explanation, verdict format	`/scan-repo`, `/scan-package`, `/explain`, `/stats`
Yash	Consumer side + threat graph	Chrome extension logic: page scan for seed-phrase phishing, wallet interception, transaction and signature decoding, intent rules, phishing checks, page-script check, contract checks; TigerGraph schema and risk queries	`/review-tx`, `/scan-contract`, `/check-scripts`, `/report`, `/graph`
Ivo	All screens + website	Shared React component library, Chrome extension screens, VS Code extension panels, Next.js website, Dodo checkout and webhook, demo sites, pitch deck	Next.js route for Dodo webhook

Decisions to confirm tonight

- [ ] Yash owns the TigerGraph schema; Imran adds Repo, Package and AlertType data through Yash's `/report` and loader functions.
- [ ] Wallet interception sits with Yash, not the frontend owner as the first plan said.
- [ ] Imran's part is the heaviest. If he falls behind, the CLI moves to Friday night and Aderyn is cut.

Shared contract

Every check returns the same verdict format, so every screen, the CLI and the graph read it the same way. Agree on this first, then build against mock responses.

Verdict levels. Never say "safe": rules find known patterns, not intent.

Level	Colour	Repo label (VS Code, banner, CLI)	Transaction label (Chrome)
`critical`	Red	Do not run	Block: reject this request
`high`	Red	Open in Restricted Mode only	High risk
`caution`	Amber	Review before running	Caution
`unknown`	Grey	Could not check fully	Not enough data
`no_known_issues`	Grey	No known issues found	No known issues found

There is no green badge anywhere.

Verdict JSON

```
{
  "level": "critical",
  "target": { "type": "repo | package | site | contract | tx", "id": "github.com/x/y@3f9c2a1" },
  "findings": [
    {
      "layer": "repo-file | dependency | secret | site | contract | tx | intent | graph",
      "severity": "critical",
      "title": "Task runs a hidden command when the folder opens",
      "evidence": ".vscode/tasks.json:14  runOn: folderOpen → curl … | sh",
      "location": { "file": ".vscode/tasks.json", "line": 14 },
      "source": "rules | socket | sourcify | graph",
      "checkedAt": "2026-10-29T18:00:00+05:30"
    }
  ],
  "indicators": { "addresses": ["0x9f…"], "domains": ["bad.example"] },
  "notChecked": ["Contract source not verified"],
  "actions": ["Do not open this folder in VS Code"],
  "explanation": "Plain-English summary written by the AI layer"
}
```

The overall level is the highest severity among the findings. One critical finding is never diluted by many harmless ones. `indicators` carries the attacker wallets and domains that go into the graph.

Endpoints

Endpoint	Owner	Used by	Purpose
`POST /scan-repo`	Imran	VS Code, CLI, banner, website	Public URL: server fetches and scans. Local folder: client sends its local findings plus the package list, no source code
`POST /scan-package`	Imran	VS Code, CLI	Package IDs → Socket + Stack3 rules
`POST /explain`	Imran	All clients	Findings → plain-English explanation. Cannot change the level
`GET /stats`	Imran	Website	Repos scanned, threats found
`POST /check-scripts`	Yash	Chrome	Script URLs a page loaded → package IDs → reuses `/scan-package` logic
`POST /review-tx`	Yash	Chrome	Decode, contract checks, intent rules, graph lookup for one request
`GET /scan-contract/:address`	Yash	Chrome, website	Verified source, proxy, owner, age
`POST /report`	Yash	VS Code, Chrome	Write attacker wallets, domains and

			packages to the graph
`GET /graph/:id`	Yash	Website	Related nodes for the graph page

Developer side: VS Code extension and CLI (Imran)

The developer side checks two things every time: the files the repo author wrote, and the packages the repo installs. The scan must finish before the folder is opened, because a malicious `.vscode/tasks.json` runs when a trusted folder opens.

Flow 1: Clone and scan safely (main flow, build first)

1. The user runs “Stack3: Clone and scan safely” and pastes a GitHub URL.
2. The extension clones the repo into a folder without opening it as the workspace, and without submodules.
3. It runs the local repo-file rules, then sends the package list to `/scan-package`.
4. It shows the verdict panel with three buttons: **Open in Restricted Mode**, **Open**, **Delete folder**.

Flow 2: Warning on an unknown folder (best effort)

The extension declares support for untrusted workspaces. When a folder with suspicious `.vscode` files or install scripts is opened directly, it shows “Don’t trust this folder” at once. It is not yet confirmed whether this appears before VS Code’s trust prompt, so treat it as an extra layer, not the main protection.

Flow 3: Guard while working

- Re-scan when `package.json` or the lockfile changes, and warn **before** `npm install`.
- Put risky lines in the editor and the Problems panel, e.g. a red underline on line 14 of `tasks.json`.
- Scan for hardcoded private keys and seed phrases, and warn before commit.
- With the user’s consent, set `"task.allowAutomaticTasks": "off"`.

What it checks

Check	How	Where it runs
Auto-run files	`.vscode/tasks.json` with `runOn: folderOpen`; terminal or shell overrides in `.vscode/settings.json`; very long lines hiding commands	Locally
Install scripts	`preinstall`, `install`, `postinstall`, `prepare` calling network tools, `node -e`, or hidden files	Locally
Source patterns	`eval`, `new Function`, base64 then execute, `child_process` in non-build files, requests to raw IPs, reads of `.env`, keystores, `~/.config/solana/id.json`	Locally
Packages	Lockfile → package IDs → Socket alerts for malware, typosquats, install scripts, env exfiltration	API
Unverified dependencies	Dependencies from a git link, a URL or a local path. Flag them, and scan their files with the same rules	Locally
Hardcoded secrets	Private keys, seed phrases and API keys committed in the repo	Locally
Contracts (stretch)	Aderyn on `.sol` files plus rules for `tx.origin`, arbitrary `delegatecall`, `selfdestruct`, owner drain functions	Locally

Report to the graph. When a finding contains an attacker wallet, domain or webhook URL, put it in `indicators` and send it to `/report`.

AI explanation. `/explain` receives only the flagged snippets and the verdict, and returns a plain-English summary and fix. It cannot change the level, so text planted in a repo cannot talk the verdict down.

CLI. `npx stack3 check <github-url>` and `npx stack3 scan ./folder`, using the same engine. Covers Cursor, terminal users and private repos: source code never leaves the machine.

Distribution. Hand out a `.vsix` file in the house. Publish to Open VSX for Cursor if time allows; the VS Code Marketplace can come after the event.

Consumer side: Chrome extension logic (Yash)

The Chrome extension (Manifest V3, EVM chains only) protects people browsing dApps and developers browsing GitHub. Ivo builds

its screens; Yash builds everything behind them.

On every signing request

1. **Intercept.** At page start, inject a script that wraps `window.ethereum` and providers announced through EIP-6963. Catch `eth_sendTransaction`, `eth_signTypedData_v4` and `personal_sign`, and forward to the wallet only after the user chooses Continue.
2. **Instant checks, under one second.** Run local rules first: blocklist, lookalike domain, decoded approval type. Show the review screen immediately.
3. **Decode.** Get the ABI from Sourcify or Etherscan and turn calldata into a readable action, falling back to the 4byte directory. Decode Permit and Permit2 typed data: spender, amount, deadline.
4. **Deeper checks.** Call `/review-tx` for contract facts, the graph lookup and intent rules. Update the screen when results arrive.
5. **Show the verdict** with reasons and two buttons: Reject, Continue anyway.

Transaction rules

Pattern	Level
Recipient or spender is in the graph as an attacker (e.g. found inside a malicious package)	`critical`
`setApprovalForAll(operator, true)` to an unknown operator	`high`
Unlimited ERC-20 approval or Permit/Permit2 to an unknown spender	`high`
Intent mismatch: button says Claim, Mint or Swap, but the request is an approval or a transfer out	`high`
`personal_sign` of raw hex or an unreadable message	`caution`

Intent rules. Read the text of the button the user clicked and the page title. Classify it with a fixed keyword table (claim, mint, swap, connect, sign in), not an LLM: page text is attacker-controlled. Compare against the decoded action.

Contract checks (`/scan-contract`): verified source; proxy and upgrade owner; whether a single wallet can upgrade with no multisig or timelock; owner powers to mint, pause, blacklist, change fees or withdraw all funds; `selfdestruct` or arbitrary `delegatecall`; contract age. Unverified source is `caution` plus a `notChecked` entry, not automatically malicious.

Page scan: does this site ask for a seed phrase?

A content script reads the whole code of every page the user opens, inside the browser, and decides whether the site is trying to collect a seed phrase or private key. It warns **when the page loads, before the user types anything**. The scan runs on the device; page code is never sent to the server.

Why scan inside the browser, not from a server crawler. Phishing kits often show a harmless page to bots and the real form only to real visitors, so a server-side scraper would miss them. The extension sees exactly what the user sees, including content added later by JavaScript.

What it reads

1. The page HTML and all visible text.
2. Every form and input: count, names, placeholders, labels, `type` attributes.
3. Inline scripts and the external scripts the page loaded.
4. Changes after load (a `MutationObserver`), because many kits inject the form a few seconds later or after a fake “Connect wallet” click.

Signals

Signal	Example	Weight
Phrase-entry layout	12, 15, 18, 21 or 24 inputs in a grid, or one large textarea	Strong
Asking words	“recovery phrase”, “seed phrase”, “secret phrase”, “mnemonic”, “private key”, “12 words”, “import wallet”, “validate wallet”, “sync wallet”	Strong
BIP39 word list in the page code	A script contains the 2,048-word BIP39 list or a phrase validator; normal dApps never need this	Strong
Where the data goes	Form posts or `fetch` calls to an unrelated domain, a Telegram bot API, a Discord webhook or a form service	Strong
Brand impersonation	Page uses a wallet brand name or logo text on a domain that is not that wallet’s official site	Medium
Domain	Lookalike of a known dApp or wallet domain,	Medium

	punycode, on a public blocklist	

Verdict rules

- Phrase-entry layout or asking words **plus** any other strong signal → `critical`: full-screen warning, page inputs disabled, user must click through on purpose.
- One strong signal alone → `high`: warning banner at the top of the page.
- Only medium signals → `caution`: badge on the extension icon.
- Allow-list: the official sites of major wallets and known wallet extension pages, so their own restore screens don't trigger warnings.

Typing guard (last line of defence). If the page scan missed it, detect 12 or 24 BIP39 words typed or pasted into any field on a non-wallet page, and block the submit with the full-screen warning.

Report to the graph. A `critical` page sends its domain and any exfiltration endpoints (Telegram bot, webhook, drain address) to `/report`, so other users and the developer side see it too. Only the domain and endpoints are sent, never the page content or anything the user typed.

Page-script check (`/check-scripts`). List the scripts the page loaded. CDN URLs such as jsDelivr include the package name and version, so they map to package IDs and go through the same Socket check. This catches the Ledger Connect Kit case: real domain, bad code.

Phishing checks. Public blocklists plus a lookalike check against a list of top dApp and wallet domains, including punycode and homoglyphs. Warn on wallet-looking downloads (`.exe`, `.dmg`, `.crx`) from unofficial sites.

GitHub repo banner. On `github.com/<owner>/<repo>` pages, call `/scan-repo` and show the verdict banner before the user clones.

Stretch: simulation. One provider (Tenderly or Alchemy) for "You send X, you receive Y". Only put that line in the demo once it works.

Limits to state honestly. Wallets that bypass browser injection (e.g. WalletConnect to a phone) are not covered; Solana is not covered in this version; code bundled into a site's own files, instead of loaded from a CDN, cannot be mapped to packages.

Frontend: screens and website (Ivo)

Ivo builds one set of React components in `packages/ui` and reuses it in three places. That is the only way one person covers the website, the Chrome extension and the VS Code extension by Saturday.

Shared components (Tailwind + shadcn/ui)

- Verdict banner: level, colour and label from the shared contract
- Findings list: title, evidence, file and line, source
- "What to do" actions block and the AI explanation box
- Graph view (react-force-graph or Cytoscape)
- Design rules: red for `critical` and `high`, amber for `caution`, grey for `unknown` and `no_known_issues`. Never a green "safe" badge.

Where they are used

Surface	Screens
Chrome extension	Review screen before signing (what you send, what you receive, who you pay, level, reasons, Reject / Continue anyway); full-screen scam and seed-phrase warning; popup; GitHub repo banner
VS Code extension	Verdict panel after "Clone and scan safely" with Open in Restricted Mode / Open / Delete folder; findings panel
Website	The pages below

Website pages (Next.js App Router, on Vercel)

Page	Content
Home	Pitch line, the two products, how the graph links them, demo GIF, install buttons for `.vsix` and the Chrome extension
Live scanner	Paste a GitHub URL, contract address or package name and get the verdict. Shows the live stats counter
Report page	Shareable link for every scan; the CLI and extensions link here
Threat graph	Package → wallet → site, from `/graph/:id`
Pricing	Free and Pro, with Dodo Payments checkout and a webhook route that marks Pro accounts

Docs	How to call `/review-tx` and `/scan-repo`

Demo sites (`apps/demo`, made-up brand, localhost or testnet only, never a copy of a real project)

- A fake wallet-recovery page asking for 12 words
- A test dApp with a "Claim reward" button that actually requests an approval to the demo attacker wallet
- The same dApp in a normal version, for the "no known issues" case

Until the API is live, every screen reads mock responses that follow the verdict JSON exactly.

Threat graph and AI layer

The graph is what makes Stack3 one product instead of two: it is the only place where a finding on the developer side meets a request on the consumer side.

Schema (TigerGraph, owned by Yash)

Vertex	Written by	Example
`Repo`	Imran	`github.com/x/interview-task@3f9c2a1`
`Package`	Imran, Yash	`pkg:npm/some-lib@1.0.3`
`AlertType`	Imran	`install-script`, `network-access`, `malware`
`Address`	Imran, Yash	Attacker wallet found in a package, or a transaction recipient
`Contract`	Yash	A dApp's contract
`Domain`	Imran, Yash	Exfiltration domain in a package, or a phishing site

Edge	From → To
`DEPENDS_ON`	Repo → Package, Package → Package
`HAS_ALERT`	Package → AlertType
`EXFILTRATES_TO`	Package or Repo → Address or Domain
`LOADS`	Domain → Package (from the page-script check)
`DEPLOYED_BY`	Contract → Address
`SENT_FUNDS_TO`, `FUNDED_BY`	Address → Address (seeded demo data only)

Risk queries

1. `tx_risk(address)`: is this recipient or spender an `EXFILTRATES_TO` target of any flagged package or repo, or deployed by one, or funded by one within two hops? If yes, `critical`.
2. `blast_radius(package)`: every repo and domain that depends on or loads this package.
3. `related(id)`: neighbours for the website's graph page.

Seeding. On day 1, load public scam addresses and the team's own demo attackers so the demo always has data. Funding links between wallets are seeded only; real chain indexing comes after the event. Say so if a judge asks.

AI layer rules

- Rules and graph queries decide the level. The AI only writes the explanation and the fix.
- The AI receives flagged snippets and the verdict, never the whole repo or page.
- Anything an attacker controls (repo files, page text) is treated as data, so instructions planted in it cannot change the verdict.

Sponsor tracks and competitors

Keep only the sponsors that posted a bounty in your onsite brief. Pitch Socket as "built on Socket's data", never as "better than Socket".

Track	How Stack3 uses it	Effort
TigerGraph	The shared threat graph and every risk query	Deep
Socket	Package threat data on both sides: lockfiles and page scripts	Deep
Dodo Payments	Pro tier checkout on the website, working in the demo	Light

Cake Wallet	One slide: `/review-tx` could run inside a wallet's confirm screen	Slide only

Competitor	Covers	What Stack3 adds
Blockaid (inside MetaMask)	Transaction scanning in the wallet	Developer side, page-script check, graph link to package attackers
Wallet Guard (Consensys), Pocket Universe (Kerberus)	Phishing and drainer warnings for users	Developer side and the graph link
Socket	Package malware detection	Repo-file checks, Web3 wallet focus, link to the consumer side
Slither, Aderyn	Solidity bugs	Package and repo malware, the consumer side
Asking Claude Code or Cursor to check a repo	Finds problems after the folder is open	Verdict before opening; rules that planted text cannot talk down; live threat data

Positioning: simulation and drainer warnings are table stakes. Lead with the shared graph, the developer-to-user link, and the Ledger story. We found no tool that links an attacker found in a developer's dependencies to a transaction a user is about to sign. Our search was limited, so we say "we did not find one", not "none exists".

Timeline to Saturday

Code freezes Saturday 31 October at 10:00 AM; submission closes at 2:00 PM IST. Each block ends with something working that the others can plug into.

Block	Imran	Yash	Ivo
Thu 6 PM – midnight	Repo set up, verdict format in `engine`, mock responses committed; lockfile → Socket; `tasks.json`, `settings.json`, install-script rules	Page scan: forms, asking words, BIP39 list detection; wallet wrapper pausing requests (checkpoint 9 PM); TigerGraph schema and seed data	`packages/ui` components on mock data; review screen; website shell
Fri 9 AM – 2 PM	VS Code "Clone and scan safely" + verdict panel; `/scan-repo`, `/scan-package` live	Page scan: late-loaded forms, data-destination check, verdict rules, typing guard; approval and Permit decoding; `/review-tx`; `/report` and `tx_risk`	Chrome screens wired to real API; VS Code panels; live scanner page
Fri 2 PM – 10 PM	Editor warnings, file watcher, secret scan, `/explain`; fake package flagged on add	Intent rules; page-script check; phishing and lookalike; GitHub banner; contract checks	Graph page; pricing and Dodo checkout; demo sites
Fri 10 PM – 2 AM	CLI; Aderyn if ahead	Simulation if ahead	Demo GIF, pitch deck
Sat 6 AM – 10 AM	Bug fixes only	Bug fixes only	Rehearse twice, record backup video
Sat 10 AM – 2 PM	Submit on Devfolio: repo, video, `.vsix`, extension zip, live URL		

Checkpoint, Thursday 9 PM. If the wallet wrapper is not reliably pausing requests, Yash stops the transaction side and puts that time into the page scan, GitHub banner and page-script check. The page scan alone still proves the consumer product.

Cut order if behind (first cut first)

1. Simulation
2. Aderyn contract checks
3. CLI
4. Fake-download warning
5. Contract owner and proxy checks
6. Flow 2 (warning on an unknown folder)

Never cut: Clone and scan safely, Socket package scan, repo-file rules, the page scan, the review screen, `/report` and `tx_risk`. Those are the whole story.

Task lists

Each person works from their own list. A task is done only when its “done when” line is true. Tick tasks off in this order.

Ivo: screens and website

Thursday, 6 PM – midnight

- [ ] `packages/ui` set up with Tailwind and shadcn/ui. *Done when:* the verdict banner shows all five levels from mock JSON.
- [ ] Findings list, actions block and AI explanation box. *Done when:* they render the sample verdict JSON from the shared contract.
- [ ] Chrome review screen (send, receive, pay to, level, reasons, Reject / Continue anyway) on mock data. *Done when:* Yash can import it.
- [ ] Website shell on Vercel with Home page. *Done when:* public URL works.

Friday, 9 AM – 2 PM

- [ ] Full-screen seed-phrase warning screen, listing the evidence from the page scan. *Done when:* Yash can show it from the content script.
- [ ] VS Code verdict panel (Open in Restricted Mode / Open / Delete folder) and findings panel as webviews. *Done when:* Imran can load them.
- [ ] Live scanner page wired to `/scan-repo` and `/stats`. *Done when:* pasting a public repo shows a real verdict.

Friday, 2 PM – 10 PM

- [ ] Report page with shareable link per scan.
- [ ] Threat graph page from `/graph/:id`. *Done when:* package → wallet → site shows for the seeded demo attacker.
- [ ] Pricing page with Dodo Payments checkout and the webhook route marking Pro accounts.
- [ ] Demo sites on a made-up brand: fake recovery page (24-word grid, bundled BIP39 list, form to a dummy endpoint), test dApp “Claim reward” requesting an approval, normal dApp.

Friday night and Saturday

- [ ] Demo GIF, pitch deck, backup video. Rehearse the demo twice by 10 AM Saturday.

Imran: developer product (VS Code extension, CLI, shared engine)

Thursday, 6 PM – midnight

- [ ] Monorepo set up; verdict types in `packages/engine`; mock responses for every endpoint committed. *Done when:* Yash and Ivo build against them.
- [ ] Lockfile parser (`package-lock.json` first) → package IDs → Socket batch call. *Done when:* a real repo returns alerts.
- [ ] Repo-file rules: `tasks.json` with `runOn: folderOpen`, `settings.json` overrides, install scripts, very long lines. *Done when:* the demo interview repo returns `critical`.

Friday, 9 AM – 2 PM

- [ ] VS Code "Clone and scan safely": clone without opening, no submodules, scan, show Ivo's verdict panel. *Done when:* the demo repo is stopped before it opens.
- [ ] `/scan-repo` and `/scan-package` live on the API.
- [ ] Indicators (wallets, domains, webhooks) extracted from findings and sent to `/report`.

Friday, 2 PM – 10 PM

- [ ] Editor and Problems panel warnings on risky lines; file watcher on `package.json` and lockfile.
- [ ] Fake demo package flagged when added to `package.json`, before `npm install`. Package stays local.
- [ ] Secret scan for private keys and seed phrases in the repo.
- [ ] `/explain`: snippets + verdict in, plain-English summary out; cannot change the level.
- [ ] `/stats` for the website counter.
- [ ] `.vsix` built for handing out.

Friday night (only if ahead)

- [ ] CLI: `npx stack3 check <url>` and `npx stack3 scan ./folder`.
- [ ] Aderyn contract checks.

Yash: consumer product (Chrome extension logic, threat graph)

Thursday, 6 PM – midnight

- [ ] Manifest V3 extension skeleton with a content script on every page.
- [ ] Page scan, first pass: count inputs and grids, find asking words, detect the BIP39 word list in page scripts. *Done when:* Ivo's fake recovery page is flagged on load.
- [ ] Wallet wrapper on `window.ethereum` and EIP-6963 providers pausing `eth_sendTransaction`, `eth_signTypedData_v4`, `personal_sign`. **Checkpoint 9 PM.**
- [ ] TigerGraph schema created; seed data with public scam addresses and the demo attacker.

Friday, 9 AM – 2 PM

- [ ] Page scan, full: `MutationObserver` for late forms; data-destination check (unrelated domain, Telegram bot, webhook); verdict rules; wallet allow-list. *Done when:* the normal dApp shows no warning and the fake page shows `critical`.
- [ ] Typing guard: block submit of 12 or 24 BIP39 words on non-wallet pages.
- [ ] Approval, `setApprovalForAll`, Permit and Permit2 decoding.
- [ ] `/review-tx`, `/report` and the `tx_risk` query. *Done when:* a request to the demo attacker wallet returns `critical` with "linked to a malicious package".

Friday, 2 PM – 10 PM

- [ ] Intent rules (button text vs decoded action) with the fixed keyword table.
- [ ] Page-script check: CDN script URLs → package IDs → `/check-scripts`.
- [ ] Phishing blocklist and lookalike-domain check.
- [ ] GitHub repo banner calling `/scan-repo`.
- [ ] `/scan-contract`: verified source, proxy, owner, age.
- [ ] Extension zip built for the submission.

Friday night (only if ahead)

- [ ] Simulation with one provider.

Demo script and traction

The demo follows one attacker wallet from a developer's machine to a user's wallet. Every step runs on demo material the team built: local only, made-up brand, nothing harmful.

Demo script (about 4 minutes)

1. **Open with the Ledger story** (20 seconds): real websites, hijacked code, users drained.
2. **VS Code, Clone and scan safely** on a demo "interview task" repo. Verdict: Do not run. `tasks.json` line 14 runs a hidden command on folder open; an install script reads `.env`. The demo attacker wallet is extracted and sent to `/report`.
3. **VS Code, add the fake package** to `package.json`. The warning appears **before** `npm install` runs. The package is a local file dependency, never published to npm.
4. **Chrome, fake recovery page** on a made-up brand. The page scan fires **as soon as the page loads**, before anything is typed: full-screen warning naming the evidence (24-word grid, BIP39 list in the page code, form posting to a Telegram bot). Then show the typing guard blocking a pasted phrase.
5. **Chrome, test dApp.** Click "Claim reward". The request is an approval to the demo attacker wallet. The review screen shows the decoded action, the intent mismatch, and: "Recipient is linked to a malicious package found in a developer's repo." Reject.
6. **The no-known-issues case.** The normal version of the same dApp shows no warning, so judges see the tool doesn't flag everything.
7. **Website graph page:** package → wallet → site, connected.
8. **Live numbers:** repos scanned in the house, real findings, Pro checkout through Dodo.

Traction plan

- From Friday 2 PM, walk the house and scan every team's repo with them using the live scanner. Target 100 repos.
- Share the report link in the event chat; every report links back to the website.
- Share findings privately first. Show a team's repo on stage only with their permission.
- Hand out the `.vsix` and CLI to anyone who wants it; don't push people to install an unpacked Chrome extension.

Risks and rules

These rules apply to all three people and every surface.

Risk	Rule

Scanning too late	Developer verdicts come before a folder is opened or `npm install` runs. A warning after the payload ran is a failure
False positives	Combine signals before `high` or `critical`. Test on normal sites and repos, including wallet docs and tutorials
Slow signing screen	Local checks first, under one second; deeper results update the screen afterwards
AI talked into a wrong verdict	Rules and graph queries set the level. The AI only explains, and repo or page text is treated as data
Page scan false alarms	Wallet allow-list, and `critical` only when two strong signals agree. Test on wallet docs, tutorials and real wallet restore pages
Privacy	The page scan, seed-phrase detection and repo-file rules run locally; page code and typed text never leave the device. Only package IDs, addresses, decoded requests and flagged snippets reach the server. Say this on the website
Overclaiming	Say “no known issues found”, never “safe”. Say “we did not find a tool that…”, never “no one else does this”
Demo material	Fake package stays local and is never published. Demo sites use a made-up brand on localhost or a testnet. Never copy a real project’s site. Never run a real malware sample on a laptop at the event
Third-party code	Check Wallet Guard’s licence before reusing any of its code
Scope	If short on time, follow the cut order in the timeline. The full story needs only: Clone and scan safely, Socket scan, repo-file rules, review screen, `/report` and `tx_risk`